

Relatório de Auditoria de Segurança — Escola Atenta

Cinco categorias • evidências verificadas • rascunhos de issues

08 de setembro de 2026

Revisão: f7550204891aa6f2914a9280fe26ddb2d55ef793

Branch: fix/revisao-sync-seguranca-dados

Escopo e método

API ASP.NET Core 9, handlers MediatR, EF Core/SQLite, app React Native/Expo/WatermelonDB e configuração de deploy Windows. Revisão do código atual, histórico Git local e sourcemap disponível. Reproduções somente com dados sintéticos e SQLite em memória.

Mapeamento das categorias: isolamento = UsuarioTurmas e cache offline; permissões = JWT, atributos Authorize e regras nos handlers; IDOR = IDs de rotas/body/SyncLog; segredos = configuração, histórico e artefatos; XSS = entrada de dados até destinos HTML/JavaScript ou UI nativa.

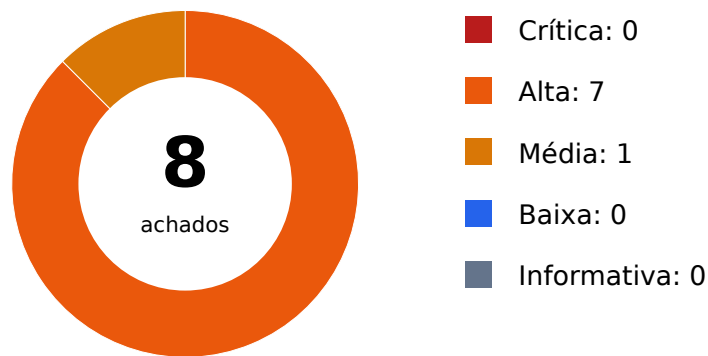
Foram inventariadas 28 ações HTTP e 29 classes de handler, mais operações diretas de autenticação e workers. Estado inicial limpo. O produto não foi modificado; issues não foram publicadas.

Leitura dos resultados

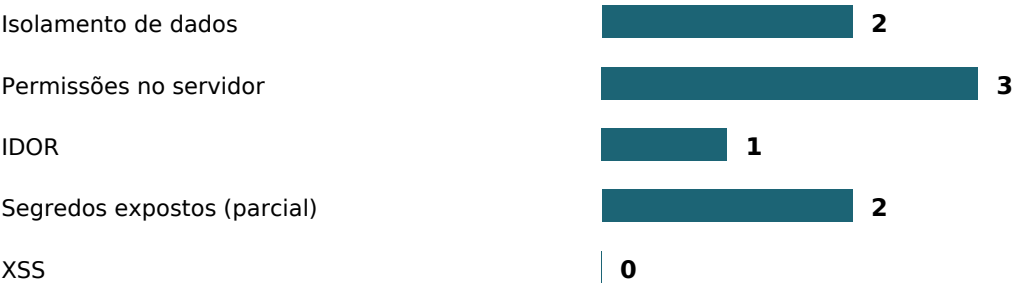
Há achados reproduzidos e achados estáticos condicionais. As condições de cada um são explícitas. Ausência de achado não equivale a prova de segurança; as lacunas constam em Cobertura e limitações.

Resumo executivo

8 achados verificados: 0 crítica, 7 alta, 1 média, 0 baixa, 0 informativa.



Achados por categoria



Risco central: o escopo por turma não é uniforme entre consultas, sync e escritas. Além disso, alterações de segurança da conta não encerram todas as sessões. Priorizar os controles compartilhados e a validação por objeto.

dotnet test: 63 domínio + 204 aplicação = 267 aprovados; 0 falhas. 15 cenários/controles do projeto isolado confirmados. Avisos existentes CS8604 e CS0618.

As contagens excluem riscos latentes e consultas cuja exploração não foi confirmada. EA-07 depende de Development; EA-08 depende das condições de instalação descritas.

Pontos fortes e pontos fracos

Gestão de usuários exige Administrador no servidor; Monitor recebeu 403.

src/EscolaAtenta.API/Controllers/UsuariosController.cs:18

Atualização de presença autoriza pela turma real do registro persistido, não pelo Turmald do cliente.

src/EscolaAtenta.Application/Chamadas/Handlers/SyncPushHandler.cs:622-639

Transferência valida vínculo com origem e destino para usuário não administrador.

src/EscolaAtenta.Application/Alunos/Handlers/TransferirAlunoHandler.cs:42-57

Migração em lote exige Administrador também no handler.

src/EscolaAtenta.Application/Turmas/Handlers/MigrarTurmaHandler.cs:29-31

Histórico de presenças restringe registros às turmas vinculadas.

src/EscolaAtenta.Application/Alunos/Handlers/GetHistoricoPresencasAlunoQueryHandler.cs:54-89

Senhas verificadas e armazenadas com BCrypt; não são retornadas nos DTOs de consulta.

src/EscolaAtenta.Infrastructure/Services/AuthService.cs:89-108

Produção sem chave gera segredo aleatório de 64 bytes; ressalva de armazenamento em EA-08.

src/EscolaAtenta.API/Program.cs:87-101

Tokens são armazenados no SecureStore; o cache escolar exige segregação adicional.

src/EscolaAtenta.App/src/services/api.ts:105-119

Pontos fracos prioritários

Leituras sem escopo de turma (EA-01), cache sem separação de conta (EA-02), regras distintas entre REST e sync (EA-03), resolução de alerta sem vínculo (EA-04) e ciclo de vida de sessão incompleto (EA-05/EA-06). Segredos de desenvolvimento e permissões locais ampliam o risco nas condições de EA-07/EA-08.

EA-01 | Consultas e pull expõem dados de turmas sem vínculo

ALTA

Categoria 1 | Isolamento de dados

Um usuário autenticado pode pedir o primeiro pull, consultar alunos com faltas ou listar turmas. Esses handlers não recebem a identidade para restringir UsuarioTurmas. AppDbContext aplica somente Ativo, não propriedade. A consulta opcional por TurmaId no dashboard é filtro do cliente, não autorização.

Pré-condições e explorabilidade

Monitor ou Supervisão autenticado, com vínculo apenas à turma A, e dados existentes na turma B. Acesso à API local da escola.

Impacto

Exposição de nomes, IDs, turma e contadores de frequência; dashboard também retorna matrícula. Permite descobrir IDs utilizados nas outras falhas. Não depende de vários tenants no mesmo banco.

Severidade	Arquivo:linha	Descrição
alta	src/EscolaAtenta.Application/Chamadas/Handlers/SyncPullHandler.cs:54-58	EA-01
alta	src/EscolaAtenta.Application/Chamadas/Handlers/SyncPullHandler.cs:114-141	EA-01
alta	src/EscolaAtenta.Application/Alunos/Queries/GetAlunosComFaltasQuery.cs:43-80	EA-01
alta	src/EscolaAtenta.Application/Turmas/Handlers/GetTurmasQueryHandler.cs:18-24	EA-01
alta	src/EscolaAtenta.API/Controllers/SyncController.cs:17-38	EA-01
alta	src/EscolaAtenta.API/Controllers/DashboardController.cs:16-38	EA-01

Trecho mínimo representativo

```
var todosAlunos = await _context.Alunos
    .AsNoTracking().ToListAsync(ct);
```

Verificação

Reprodução HTTP com SQLite em memória: pull, dashboard e listagem retornaram 200 e incluíram os dados da turma B para Monitor vinculado apenas à A.

Correção e aceite

Aplicar escopo comum por UsuarioTurmas às consultas e aos deltas created/updated/deleted; permitir abrangência global apenas a Administrador. Definir remoção local quando um vínculo for revogado.

Critérios de aceite específicos no rascunho de issue correspondente, ao final do relatório.

EA-02 | Cache offline permanece compartilhado entre contas e servidores

ALTA

Categoria 1 | Isolamento de dados

O app mantém uma única instância de WatermelonDB. Sair remove tokens e estado React, mas não associa nem segrega o banco por usuário/servidor. Após outro login, telas consultam o mesmo cache por turma; a configuração de outro servidor apenas troca a URL.

Pré-condições e explorabilidade

Mesma instalação do app já contém dados; outra conta autenticada entra, ou o servidor configurado é alterado. Não exige root no Android. A revisão é estática, sem execução em dispositivo.

Impacto

A segunda sessão pode visualizar dados baixados pela anterior. Deltas locais pendentes também não carregam proprietário de sessão, criando risco de envio sob a identidade seguinte. O defeito permanece mesmo após corrigir o filtro do servidor.

Severidade	Arquivo:linha	Descrição
alta	src/EscolaAtenta.App/src/contexts/AuthContext.tsx:98-102	EA-02
alta	src/EscolaAtenta.App/src/database/index.ts:10-24	EA-02
alta	src/EscolaAtenta.App/src/screens/gestao/AlunosScreen.tsx:36-57	EA-02
alta	src/EscolaAtenta.App/src/services/serverConfig.ts:10-13	EA-02
alta	src/EscolaAtenta.App/src/services/sync/watermelondbSync.ts:165-178	EA-02

Trecho mínimo representativo

```
async function signOut() {  
    await authStorage.removeToken();  
    setUser(null);  
    setDeveAlterarSenha(false);  
}
```

Verificação

Fluxo estático completo: singleton database -> signOut sem segregação -> nova sessão -> consulta local apenas por turma_id. Não houve reprodução Android.

Correção e aceite

Segregar cache e fila de sync por servidor e identidade, suspendendo sync durante a troca. Preservar pendências da conta anterior em armazenamento inacessível à nova sessão; evitar apagá-las indiscriminadamente.

Critérios de aceite específicos no rascunho de issue correspondente, ao final do relatório.

EA-03 | Rotas REST contornam a restrição administrativa aplicada no sync

ALTA

Categoria 2 | Permissões no servidor

O sync permite criar e editar cadastros apenas a Administrador, mas as rotas REST exigem só autenticação. As criações não verificam papel; criar aluno verifica apenas se TurmaId existe. As edições aceitam qualquer papel com vínculo, contrariando o bloqueio administrativo equivalente do sync.

Pré-condições e explorabilidade

Monitor ou Supervisão com token válido. Para criar aluno em turma alheia basta conhecer seu GUID; ele é obtível em EA-01. Edições exigem vínculo, ao contrário das criações.

Impacto

Criação de turmas e inserção de alunos em turmas não autorizadas, além de edição de cadastros vinculados por papéis que o sync proíbe. Compromete integridade cadastral.

Severidade	Arquivo:linha	Descrição
alta	src/EscolaAtenta.Application/Turmas/Handlers/CriarTurmaHandler.cs:18-28	EA-03
alta	src/EscolaAtenta.Application/Alunos/Handlers/CriarAlunoHandler.cs:19-37	EA-03
alta	src/EscolaAtenta.Application/Turmas/Handlers/AtualizarTurmaHandler.cs:35-51	EA-03
alta	src/EscolaAtenta.Application/Alunos/Handlers/AtualizarAlunoHandler.cs:35-51	EA-03
alta	src/EscolaAtenta.API/Controllers/TurmasController.cs:14-35	EA-03
alta	src/EscolaAtenta.API/Controllers/AlunosController.cs:13-32	EA-03
alta	src/EscolaAtenta.Application/Chamadas/Handlers/SyncPushHandler.cs:172-183	EA-03
alta	src/EscolaAtenta.Application/Chamadas/Handlers/SyncPushHandler.cs:685-695	EA-03

Trecho mínimo representativo

```
var turma = await _context.Turmas
    .FirstOrDefaultAsync(t => t.Id == request.TurmaId, cancellationToken);

// Apenas existência é verificada antes de criar o aluno.
```

Verificação

HTTP 201 para Monitor criar turma e inserir aluno na turma B sem vínculo. Edições verificadas estaticamente. Agrupamento por causa raiz: políticas diferentes para a mesma operação.

Correção e aceite

Centralizar a política administrativa e aplicá-la igualmente no REST e no sync. Se a política permitir criação delegada, verificar explicitamente o vínculo da turma e documentar a exceção.

Critérios de aceite específicos no rascunho de issue correspondente, ao final do relatório.

EA-04 | Supervisão resolve alertas de turmas sem vínculo

ALTA

Categoria 3 | IDOR

A rota restringe o papel, mas o handler busca qualquer AlertaId e verifica somente se o identificador do chamador é um GUID. Não compara o TurmaId do alerta com UsuarioTurmas.

Pré-condições e explorabilidade

Usuário de Supervisão, GUID de um alerta de outra turma e justificativa válida. O GUID deve ser conhecido previamente; não se presume adivinhação de GUIDs.

Impacto

Encerramento indevido de alertas de evasão e gravação de justificativa/responsável fora da área de atuação. O Monitor é bloqueado pela rota, mas Supervisão não tem acesso global nas consultas protegidas.

Severidade	Arquivo:linha	Descrição
alta	src/EscolaAtenta.Application/Alertas/Handlers/ResolverAlertaHandler.cs:20-37	EA-04
alta	src/EscolaAtenta.API/Controllers/AlertasController.cs:74-81	EA-04

Trecho mínimo representativo

```
var alerta = await _context.AlertasEvasao
    .FirstOrDefaultAsync(a => a.Id == request.AlertId, cancellationToken);
alerta.MarcarComoResolvido(usuarioId, request.Justificativa);
```

Verificação

Com Supervisor vinculado apenas à A, PATCH de alerta da B retornou 204 no host isolado com SQLite.

Correção e aceite

Autorizar pela turma persistida no alerta antes de MarcarComoResolvido. Administrador pode ter acesso global; definir tratamento restrito para alertas sem turma.

Critérios de aceite específicos no rascunho de issue correspondente, ao final do relatório.

EA-05 | Troca obrigatória de senha é imposta apenas pela navegação

MÉDIA

Categoria 2 | Permissões no servidor

O login emite JWT com papel completo quando DeveAlterarSenha=true. Nenhuma política do servidor restringe esse token. O app decide mostrar a tela de troca por estado React; a restauração de sessão não restaura essa obrigação e a resposta de refresh usa o valor padrão false.

Pré-condições e explorabilidade

Credencial inicial válida de uma conta marcada para troca obrigatória. O atacante não obtém a senha por esta falha; ele contorna a restrição depois de autenticar.

Impacto

Uso completo da conta com senha inicial, inclusive funções administrativas para uma conta Administrador, sem concluir a troca exigida.

Severidade	Arquivo:linha	Descrição
média	src/EscolaAtenta.App/src/navigation/AppNavigator.tsx:45-53	EA-05
média	src/EscolaAtenta.App/src/contexts/AuthContext.tsx:42-65	EA-05
média	src/EscolaAtenta.Application/Auth/LoginHandler.cs:58-82	EA-05
média	src/EscolaAtenta.API/Controllers/AuthController.cs:142-150	EA-05
média	src/EscolaAtenta.Infrastructure/Services/AuthService.cs:48-73	EA-05

Trecho mínimo representativo

```
var loginResult = _authService.GerarToken(usuario);  
  
// DeveAlterarSenha é apenas um campo da resposta.
```

Verificação

Login retornou deveAlterarSenha=true e o JWT foi aceito em GET /usuarios com 200 antes de qualquer troca.

Correção e aceite

Aplicar bloqueio no servidor para contas com troca pendente, permitindo somente troca e encerramento da sessão. Revalidar a obrigação na restauração/refresh e emitir sessão adequada após a troca.

Critérios de aceite específicos no rascunho de issue correspondente, ao final do relatório.

EA-06 | Mudanças de segurança não invalidam sessões existentes

ALTA

Categoria 2 | Permissões no servidor

A validação JWT verifica assinatura, emissor, público e prazo, mas não o estado/papel atual no banco nem uma versão de sessão. Desativação e rebaixamento alteram apenas Usuario. Troca de senha não revoga refresh tokens.

Pré-condições e explorabilidade

Posse de token emitido antes da alteração. O JWT persiste até expirar, por padrão 60 minutos e tolerância de 30 segundos. Refresh anterior à troca de senha permanece utilizável por até seu prazo de 30 dias; login posterior revoga os anteriores.

Impacto

Conta desativada continua lendo dados; ex-Administrador continua acessando endpoints administrativos até o JWT expirar. Um refresh comprometido pode restabelecer sessão após a vítima trocar a senha.

Severidade	Arquivo:linha	Descrição
alta	src/EscolaAtenta.API/Program.cs:112-135	EA-06
alta	src/EscolaAtenta.Application/Usuarios/Commands/AlternarStatusUsuarioCommand.cs:23-33	EA-06
alta	src/EscolaAtenta.Application/Usuarios/Commands/AtualizarUsuarioCommand.cs:21-27	EA-06
alta	src/EscolaAtenta.API/Controllers/AuthController.cs:102-107	EA-06
alta	src/EscolaAtenta.API/Controllers/AuthController.cs:123-142	EA-06
alta	src/EscolaAtenta.Infrastructure/Services/AuthService.cs:46-57	EA-06

Trecho mínimo representativo

```
usuario.AlterarSenha(novoHash);  
  
await _dbContext.SaveChangesAsync(ct);  
  
// Não revoga RefreshTokens nem invalida JWTs existentes.
```

Verificação

Três reproduções: JWT de usuário inativo aceito; papel Administrador antigo aceito após rebaixamento; refresh anterior aceito após trocar senha. Todos retornaram 200.

Correção e aceite

Introduzir versão de sessão/security stamp validada no servidor; revogar sessões e refresh tokens em desativação, rebaixamento e troca de senha. Manter os fluxos legítimos de refresh e offline conforme política explícita.

Critérios de aceite específicos no rascunho de issue correspondente, ao final do relatório.

EA-07 | Fallback JWT conhecido permite forjar Administrador em Development

ALTA

Categoria 4 | Segredos expostos

Quando `Jwt:SecretKey` está vazia em Development, emissor e validador usam o mesmo literal versionado. A configuração atual de desenvolvimento deixa a chave vazia, e o perfil de execução escuta em 0.0.0.0. Uma pessoa com acesso ao código consegue assinar JWTs com papel Administrador.

Pré-condições e explorabilidade

API em Development com chave vazia e alcançável pelo atacante. Este achado é condicional ao modo; produção com chave vazia gera 64 bytes aleatórios, portanto não usa esse fallback.

Impacto

Autenticação forjada e acesso administrativo sem senha. A assinatura local foi demonstrada com sub sintético e papel Administrador, sem publicar chave ou token.

Severidade	Arquivo:linha	Descrição
alta	src/EscolaAtenta.API/Program.cs:79-84	EA-07
alta	src/EscolaAtenta.Infrastructure/Services/AuthService.cs:32-40	EA-07
alta	src/EscolaAtenta.API/Properties/launchSettings.json:8-10	EA-07
alta	src/EscolaAtenta.API/appsettings.Development.json:33-37	EA-07

Trecho mínimo representativo

```
if (builder.Environment.IsDevelopment()) {  
    secretKey = "[SEGREDO REDIGIDO]";  
}
```

Verificação

Validação criptográfica local confirmou papel Administrador. Histórico: commit 1fadfb0da5698028b8ac21ca9194e3164b42f3e9, src/EscolaAtenta.API/appsettings.Development.json:34 e build-temp/appsettings.Development.json:34 continham chave literal redigida.

Correção e aceite

Remover o literal de ambos os caminhos. Usar segredo local aleatório ou exigir configuração segura; restringir a interface de desenvolvimento a loopback. Rotacionar chaves de instâncias que tenham usado o literal.

Critérios de aceite específicos no rascunho de issue correspondente, ao final do relatório.

EA-08 | Instalador concede leitura de segredos locais a usuários comuns

ALTA

Categoria 4 | Segredos expostos

O instalador concede users-readexec à pasta base; no Inno Setup essa permissão inclui herança para descendentes. O startup de produção grava a chave JWT em API/appsettings.json e o seed registra a senha inicial em logs sob a pasta base. Não há ACL restritiva específica nesses destinos no instalador.

Pré-condições e explorabilidade

Instalação pelo script fornecido, herança padrão mantida e conta Windows comum com acesso à máquina. Chave gerada em arquivo e/ou logs do seed presentes. ACL de uma instalação real não foi inspecionada; é achado condicional do artefato de deploy.

Impacto

Leitura da chave de assinatura por usuário local permite forjar identidade administrativa; logs podem expor credencial inicial. A aleatoriedade da chave não protege contra sua leitura.

Severidade	Arquivo:linha	Descrição
alta	escolaatenta-installer.iss:66-70	EA-08
alta	src/EscolaAtenta.API/Program.cs:87-101	EA-08
alta	src/EscolaAtenta.Infrastructure/Data/DatabaseSeeder.cs:100-105	EA-08
alta	src/EscolaAtenta.API/appsettings.json:12-12	EA-08

Trecho mínimo representativo

```
Name: "{app}"; Permissions: admins-full system-full users-readexec
```

```
json["Jwt"]!["SecretKey"] = secretKey;
```

Verificação

Evidência estática do script e dos destinos de gravação; sem instalação, UAC ou alteração de ACL na máquina. Sem comprovação dinâmica de uma instalação específica.

Correção e aceite

Separar dados, segredos e logs dos binários. Restringir leitura ao serviço/SYSTEM e administradores. Dar ao TrayMonitor somente a informação operacional necessária, sem acesso à chave JWT. Provisionar a senha inicial por canal restrito e temporário.

Referência técnica: Inno Setup, [Dirs] / Permissions / readexec: https://jrsoftware.org/ishelp/topic_dirssection.htm

Critérios de aceite específicos no rascunho de issue correspondente, ao final do relatório.

Cobertura e limitações

Categoria	Resultado
Isolamento de dados	Revisada com achados; cache validado estaticamente
Permissões no servidor	Revisada com achados
IDOR	Revisada com achados; todos os handlers inventariados
Segredos expostos	Parcial: histórico e fonte; APK/bytecode/ACL real não validados
XSS	Revisada sem achados no código atual; sem teste dinâmico

Inventário completo

Consulte cobertura.md para cada handler/operação, arquivo:linha, origem dos IDs, controle efetivo e resultado. dados.json contém 33 entradas detalhadas de cobertura e as 28 rotas HTTP. inventario.json registra os 705 arquivos textuais pesquisados, hashes e resultados redigidos; inventário não significa leitura manual integral de cada arquivo.

- Auditoria das cinco categorias solicitadas, baseada em código e reproduções locais. Não é pentest de produção, auditoria de dependências/CVEs, rede escolar, Android nativo ou OTA completo.
- Estado inicial limpo no Git; commit f7550204891aa6f2914a9280fe26ddb2d55ef793, branch fix/revisao-sync-seguranca-dados. Nenhum arquivo de produto foi corrigido.
- Host de reprodução registra os controllers/handlers reais e autenticação equivalente, com SQLite em memória. Omite rate limiter, workers, startup e middleware global de erros; não é teste do Program.cs completo. Erro 500 de controle negado nesse host não é o status esperado de produção.
- Histórico: 85 commits alcançáveis por refs locais e 1.821 blobs candidatos únicos examinados por heurística; sem fetch, reflog, objetos inalcançáveis ou teste remoto de credenciais. Binários, arquivos grandes e dependências têm cobertura limitada. .env contém exemplo legado; scanner não cobre todas as atribuições sem aspas.
- Sourcemap local: 1.473 fontes e 52 entradas fora de node_modules; sem assinaturas de credenciais no teste limitado. Artefato pode estar desatualizado em relação ao HEAD. APKs e bytecode Hermes localizados não foram decompilados, e nenhum novo bundle foi gerado.
- GetAuditoriaAlertasQueryHandler não filtra vínculos, mas a consulta de detalhes falhou com SQLite devido à ordenação DateTimeOffset. Não foi contado como vazamento comprovado; corrigir autorização junto da compatibilidade do provider. Os testes existentes usam InMemory nesse handler.
- RegistrarPresencaHandler não tem rota nem chamador de produto encontrado. A associação AlunoId/ChamadaId não é validada nesse caminho interno; risco latente fora da contagem de IDOR explorável.
- ObterHistoricoTurmasAlunoHandler permite histórico completo após vínculo com qualquer turma histórica, conforme comentário explícito. A política de negócio dessa abrangência deve ser confirmada; não foi considerada falha comprovada.
- Não há filtro global EscolaId, mas a implementação atual é single-school por banco e CloudEgressWorker é stub. Não foi inventado cenário ativo de isolamento multiescola. AGENTS.md descreve filtros multi-tenant que o código atual não implementa.
- Alguns guards dependem de Guid.TryParse e não negam identidade malformada; o emissor legítimo só emite GUID. Sem caminho independente demonstrado para obter token inválido assinado, não há achado adicional.
- XSS: não foi identificado fluxo de entrada controlada para execução HTML/JavaScript no código atual. A UI usa componentes React Native Text e a API retorna JSON. Menções a WebView em lockfile não provam sink. Nenhum teste em navegador/Android foi executado.
- EA-02 e EA-08 têm comprovação estática; cache não foi executado no Android e ACL não foi testada em instalação real. Transporte HTTP local é configuração declarada; não foi reclassificado como chave hardcoded ou como XSS.

Recomendações priorizadas

P1 | EA-01, EA-03, EA-04

Unificar autorização por papel e por turma em todas as superfícies da API, incluindo pull e operações REST.

P1 | EA-06

Revogar sessões em mudanças de segurança e impedir uso de privilégios antigos.

P1 condicional | EA-07, EA-08

Eliminar a chave conhecida de Development e restringir leitura de segredos na instalação Windows; rotacionar instâncias afetadas.

P2 | EA-02

Segregar armazenamento e fila offline por servidor e identidade, preservando pendências.

P2 | EA-05

Aplicar a troca obrigatória no servidor e restaurar o estado correto no app.

Prioridade indica ordem de execução; severidade indica impacto e explorabilidade. Validar em ambiente sintético: negativo sem vínculo/papel, positivo autorizado, mesma operação online/offline e mudança de identidade com pendências. Não publicar os rascunhos sem revisão do responsável.

Regeneração

python docs/security-audit/gerar_relatorio.py. Dados, fontes e rascunhos são locais. README.md documenta dependências, comandos e limitações. O script de geração não consulta o repositório nem a internet.

ISSUES PARA O GITHUB

Rascunhos em Markdown literal, também disponíveis em issues.md. Nenhuma issue foi publicada. Os blocos abaixo são selecionáveis; quebras visuais de linha podem ser recompostas ao copiar.

--- ISSUE 1 ---

[Segurança] Consultas e pull expõem dados de turmas sem vínculo

Labels sugeridas: security, alta

Descrição

EA-01: Um usuário autenticado pode pedir o primeiro pull, consultar alunos com faltas ou listar turmas. Esses handlers não recebem a identidade para restringir UsuarioTurmas. AppDbContext aplica somente Ativo, não propriedade. A consulta opcional por TurmaId no dashboard é filtro do cliente, não autorização.

Pré-condições: Monitor ou Supervisão autenticado, com vínculo apenas à turma A, e dados existentes na turma B. Acesso à API local da escola.

Evidência

- Arquivo: src/EscolaAtenta.Application/Chamadas/Handlers/SyncPullHandler.cs:54-58
- Arquivo: src/EscolaAtenta.Application/Chamadas/Handlers/SyncPullHandler.cs:114-141
- Arquivo: src/EscolaAtenta.Application/Alunos/Queries/GetAlunosComFaltasQuery.cs:43-80
- Arquivo: src/EscolaAtenta.Application/Turmas/Handlers/GetTurmasQueryHandler.cs:18-24
- Arquivo: src/EscolaAtenta.API/Controllers/SyncController.cs:17-38
- Arquivo: src/EscolaAtenta.API/Controllers/DashboardController.cs:16-38
- Revisão: f7550204891aa6f2914a9280fe26ddb2d55ef793

```text

```
var todosAlunos = await _context.Alunos
 .AsNoTracking().ToListAsync(ct);
```

```

Verificação: Reprodução HTTP com SQLite em memória: pull, dashboard e listagem retornaram 200 e incluíram os dados da turma B para Monitor vinculado apenas à A.

Impacto

Exposição de nomes, IDs, turma e contadores de frequência; dashboard também retorna matrícula. Permite descobrir IDs utilizados nas outras falhas. Não depende de vários tenants no mesmo banco.

Sugestão de correção

Aplicar escopo comum por UsuarioTurmas às consultas e aos deltas created/updated/deleted; permitir abrangência global apenas a Administrador. Definir remoção local quando um vínculo for revogado.

Critérios de aceite

- [] Reproduzir o cenário descrito e confirmar a rejeição ou segregação esperada.
- [] Preservar o fluxo legítimo para o papel e os vínculos autorizados.
- [] Adicionar teste de regressão que use os controles reais e SQLite quando aplicável.

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] Cache offline permanece compartilhado entre contas e servidores

Labels sugeridas: security, alta

Descrição

EA-02: O app mantém uma única instância de WatermelonDB. Sair remove tokens e estado React, mas não associa nem segrega o banco por usuário/servidor. Após outro login, telas consultam o mesmo cache por turma; a configuração de outro servidor apenas troca a URL.

Pré-condições: Mesma instalação do app já contém dados; outra conta autenticada entra, ou o servidor configurado é alterado. Não exige root no Android. A revisão é estática, sem execução em dispositivo.

Evidência

- Arquivo: src/EscolaAtenta.App/src/contexts/AuthContext.tsx:98-102
- Arquivo: src/EscolaAtenta.App/src/database/index.ts:10-24
- Arquivo: src/EscolaAtenta.App/src/screens/gestao/AlunosScreen.tsx:36-57
- Arquivo: src/EscolaAtenta.App/src/services/serverConfig.ts:10-13
- Arquivo: src/EscolaAtenta.App/src/services/sync/watermelondbSync.ts:165-178
- Revisão: f7550204891aa6f2914a9280fe26ddb2d55ef793

```text

```
async function signOut() {
 await authStorage.removeToken();
 setUser(null);
 setDeveAlterarSenha(false);
}
...
```

Verificação: Fluxo estático completo: singleton database -> signOut sem segregação -> nova sessão -> consulta local apenas por turma\_id. Não houve reprodução Android.

### ## Impacto

A segunda sessão pode visualizar dados baixados pela anterior. Deltas locais pendentes também não carregam proprietário de sessão, criando risco de envio sob a identidade seguinte. O defeito permanece mesmo após corrigir o filtro do servidor.

### ## Sugestão de correção

Segregar cache e fila de sync por servidor e identidade, suspendendo sync durante a troca. Preservar pendências da conta anterior em armazenamento inacessível à nova sessão; evitar apagá-las indiscriminadamente.

### ## Critérios de aceite

- [ ] Reproduzir o cenário descrito e confirmar a rejeição ou segregação esperada.
- [ ] Preservar o fluxo legítimo para o papel e os vínculos autorizados.
- [ ] Adicionar teste de regressão que use os controles reais e SQLite quando aplicável.

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

# [Segurança] Rotas REST contornam a restrição administrativa aplicada no sync

Labels sugeridas: security, alta

### ## Descrição

EA-03: O sync permite criar e editar cadastros apenas a Administrador, mas as rotas REST exigem só autenticação. As criações não verificam papel; criar aluno verifica apenas se TurmaId existe. As edições aceitam qualquer papel com vínculo, contrariando o bloqueio administrativo equivalente do sync.

Pré-condições: Monitor ou Supervisão com token válido. Para criar aluno em turma alheia basta conhecer seu GUID; ele é obtível em EA-01. Edições exigem vínculo, ao contrário das criações.

### ## Evidência

- Arquivo: src/EscolaAtenta.Application/Turmas/Handlers/CriarTurmaHandler.cs:18-28
- Arquivo: src/EscolaAtenta.Application/Alunos/Handlers/CriarAlunoHandler.cs:19-37
- Arquivo: src/EscolaAtenta.Application/Turmas/Handlers/AtualizarTurmaHandler.cs:35-51
- Arquivo: src/EscolaAtenta.Application/Alunos/Handlers/AtualizarAlunoHandler.cs:35-51
- Arquivo: src/EscolaAtenta.API/Controllers/TurmasController.cs:14-35
- Arquivo: src/EscolaAtenta.API/Controllers/AlunosController.cs:13-32
- Arquivo: src/EscolaAtenta.Application/Chamadas/Handlers/SyncPushHandler.cs:172-183
- Arquivo: src/EscolaAtenta.Application/Chamadas/Handlers/SyncPushHandler.cs:685-695
- Revisão: f7550204891aa6f2914a9280fe26ddb2d55ef793

```text

```
var turma = await _context.Turmas
    .FirstOrDefaultAsync(t => t.Id == request.TurmaId, cancellationToken);

// Apenas existência é verificada antes de criar o aluno.
...
```

Verificação: HTTP 201 para Monitor criar turma e inserir aluno na turma B sem vínculo. Edições verificadas estaticamente. Agrupamento por causa raiz: políticas diferentes para a mesma operação.

Impacto

Criação de turmas e inserção de alunos em turmas não autorizadas, além de edição de cadastros vinculados por papéis que o sync proíbe. Compromete integridade cadastral.

Sugestão de correção

Centralizar a política administrativa e aplicá-la igualmente no REST e no sync. Se a política permitir criação delegada, verificar explicitamente o vínculo da turma e documentar a exceção.

Critérios de aceite

- [] Reproduzir o cenário descrito e confirmar a rejeição ou segregação esperada.
- [] Preservar o fluxo legítimo para o papel e os vínculos autorizados.
- [] Adicionar teste de regressão que use os controles reais e SQLite quando aplicável.

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

[Segurança] Supervisão resolve alertas de turmas sem vínculo

Labels sugeridas: security, alta

Descrição

EA-04: A rota restringe o papel, mas o handler busca qualquer AlertaId e verifica somente se o identificador do chamador é um GUID. Não compara o TurmaId do alerta com UsuarioTurmas.

Pré-condições: Usuário de Supervisão, GUID de um alerta de outra turma e justificativa válida. O GUID deve ser conhecido previamente; não se presume adivinhação de GUIDs.

Evidência

- Arquivo: src/EscolaAtenta.Application/Alertas/Handlers/ResolverAlertaHandler.cs:20-37
- Arquivo: src/EscolaAtenta.API/Controllers/AlertasController.cs:74-81
- Revisão: f7550204891aa6f2914a9280fe26ddb2d55ef793

```text

```
var alerta = await _context.AlertasEvasao
 .FirstOrDefaultAsync(a => a.Id == request.AlertId, cancellationToken);
alerta.MarcarComoResolvido(usuarioId, request.Justificativa);
```
```

Verificação: Com Supervisor vinculado apenas à A, PATCH de alerta da B retornou 204 no host isolado com SQLite.

Impacto

Encerramento indevido de alertas de evasão e gravação de justificativa/responsável fora da área de atuação. O Monitor é bloqueado pela rota, mas Supervisão não tem acesso global nas consultas protegidas.

Sugestão de correção

Autorizar pela turma persistida no alerta antes de MarcarComoResolvido. Administrador pode ter acesso global; definir tratamento restrito para alertas sem turma.

Critérios de aceite

- [] Reproduzir o cenário descrito e confirmar a rejeição ou segregação esperada.
- [] Preservar o fluxo legítimo para o papel e os vínculos autorizados.
- [] Adicionar teste de regressão que use os controles reais e SQLite quando aplicável.

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

[Segurança] Troca obrigatória de senha é imposta apenas pela navegação

Labels sugeridas: security, média

Descrição

EA-05: O login emite JWT com papel completo quando DeveAlterarSenha=true. Nenhuma política do servidor restringe esse token. O app decide mostrar a tela de troca por estado React; a restauração de sessão não restaura essa obrigação e a resposta de refresh usa o valor padrão false.

Pré-condições: Credencial inicial válida de uma conta marcada para troca obrigatória. O atacante não obtém a senha por esta falha; ele contorna a restrição depois de autenticar.

Evidência

- Arquivo: src/EscolaAtenta.App/src/navigation/AppNavigator.tsx:45-53
- Arquivo: src/EscolaAtenta.App/src/contexts/AuthContext.tsx:42-65
- Arquivo: src/EscolaAtenta.Application/Auth/LoginHandler.cs:58-82
- Arquivo: src/EscolaAtenta.API/Controllers/AuthController.cs:142-150
- Arquivo: src/EscolaAtenta.Infrastructure/Services/AuthService.cs:48-73
- Revisão: f7550204891aa6f2914a9280fe26ddb2d55ef793

```text

```
var loginResult = _authService.GerarToken(usuario);

// DeveAlterarSenha é apenas um campo da resposta.
...
`
```

Verificação: Login retornou deveAlterarSenha=true e o JWT foi aceito em GET /usuarios com 200 antes de qualquer troca.

### ## Impacto

Uso completo da conta com senha inicial, inclusive funções administrativas para uma conta Administrador, sem concluir a troca exigida.

### ## Sugestão de correção

Aplicar bloqueio no servidor para contas com troca pendente, permitindo somente troca e encerramento da sessão. Revalidar a obrigação na restauração/refresh e emitir sessão adequada após a troca.

### ## Critérios de aceite

- [ ] Reproduzir o cenário descrito e confirmar a rejeição ou segregação esperada.
- [ ] Preservar o fluxo legítimo para o papel e os vínculos autorizados.
- [ ] Adicionar teste de regressão que use os controles reais e SQLite quando aplicável.

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

# [Segurança] Mudanças de segurança não invalidam sessões existentes

Labels sugeridas: security, alta

## ## Descrição

EA-06: A validação JWT verifica assinatura, emissor, público e prazo, mas não o estado/papel atual no banco nem uma versão de sessão. Desativação e rebaixamento alteram apenas Usuario. Troca de senha não revoga refresh tokens.

Pré-condições: Posse de token emitido antes da alteração. O JWT persiste até expirar, por padrão 60 minutos e tolerância de 30 segundos. Refresh anterior à troca de senha permanece utilizável por até seu prazo de 30 dias; login posterior revoga os anteriores.

## ## Evidência

- Arquivo: src/EscolaAtenta.API/Program.cs:112-135
- Arquivo: src/EscolaAtenta.Application/Usuarios/Commands/AlternarStatusUsuarioCommand.cs:23-33
- Arquivo: src/EscolaAtenta.Application/Usuarios/Commands/AtualizarUsuarioCommand.cs:21-27
- Arquivo: src/EscolaAtenta.API/Controllers/AuthController.cs:102-107
- Arquivo: src/EscolaAtenta.API/Controllers/AuthController.cs:123-142
- Arquivo: src/EscolaAtenta.Infrastructure/Services/AuthService.cs:46-57
- Revisão: f7550204891aa6f2914a9280fe26ddb2d55ef793

```text

```
usuario.AlterarSenha(novoHash);  
  
await _dbContext.SaveChangesAsync(ct);  
  
// Não revoga RefreshTokens nem invalida JWTs existentes.  
```
```

Verificação: Três reproduções: JWT de usuário inativo aceito; papel Administrador antigo aceito após rebaixamento; refresh anterior aceito após trocar senha. Todos retornaram 200.

## ## Impacto

Conta desativada continua lendo dados; ex-Administrador continua acessando endpoints administrativos até o JWT expirar. Um refresh comprometido pode restabelecer sessão após a vítima trocar a senha.

## ## Sugestão de correção

Introduzir versão de sessão/security stamp validada no servidor; revogar sessões e refresh tokens em desativação, rebaixamento e troca de senha. Manter os fluxos legítimos de refresh e offline conforme política explícita.

## ## Critérios de aceite

- [ ] Reproduzir o cenário descrito e confirmar a rejeição ou segregação esperada.
- [ ] Preservar o fluxo legítimo para o papel e os vínculos autorizados.
- [ ] Adicionar teste de regressão que use os controles reais e SQLite quando aplicável.

--- FIM ISSUE 6 ---

--- ISSUE 7 ---

# [Segurança] Fallback JWT conhecido permite forjar Administrador em Development

Labels sugeridas: security, alta

## ## Descrição

EA-07: Quando Jwt:SecretKey está vazia em Development, emissor e validador usam o mesmo literal versionado. A configuração atual de desenvolvimento deixa a chave vazia, e o perfil de execução escuta em 0.0.0.0. Uma pessoa com acesso ao código consegue assinar JWTs com papel Administrador.

Pré-condições: API em Development com chave vazia e alcançável pelo atacante. Este achado é condicional ao modo; produção com chave vazia gera 64 bytes aleatórios, portanto não usa esse fallback.

## ## Evidência

- Arquivo: src/EscolaAtenta.API/Program.cs:79-84
- Arquivo: src/EscolaAtenta.Infrastructure/Services/AuthService.cs:32-40
- Arquivo: src/EscolaAtenta.API/Properties/launchSettings.json:8-10
- Arquivo: src/EscolaAtenta.API/appsettings.Development.json:33-37
- Revisão: f7550204891aa6f2914a9280fe26ddb2d55ef793

```text

```
if (builder.Environment.IsDevelopment()) {  
    secretKey = "[SEGREDO REDIGIDO]";  
}  
...
```

Verificação: Validação criptográfica local confirmou papel Administrador. Histórico: commit 1fadfb0da5698028b8ac21ca9194e3164b42f3e9, src/EscolaAtenta.API/appsettings.Development.json:34 e build-temp/appsettings.Development.json:34 continham chave literal redigida.

Impacto

Autenticação forjada e acesso administrativo sem senha. A assinatura local foi demonstrada com sub sintético e papel Administrador, sem publicar chave ou token.

Sugestão de correção

Remover o literal de ambos os caminhos. Usar segredo local aleatório ou exigir configuração segura; restringir a interface de desenvolvimento a loopback. Rotacionar chaves de instâncias que tenham usado o literal.

Critérios de aceite

- [] Reproduzir o cenário descrito e confirmar a rejeição ou segregação esperada.
- [] Preservar o fluxo legítimo para o papel e os vínculos autorizados.
- [] Adicionar teste de regressão que use os controles reais e SQLite quando aplicável.

--- FIM ISSUE 7 ---

--- ISSUE 8 ---

[Segurança] Instalador concede leitura de segredos locais a usuários comuns

Labels sugeridas: security, alta

Descrição

EA-08: O instalador concede users-readexec à pasta base; no Inno Setup essa permissão inclui herança para descendentes. O startup de produção grava a chave JWT em API/appsettings.json e o seed registra a senha inicial em logs sob a pasta base. Não há ACL restritiva específica nesses destinos no instalador.

Pré-condições: Instalação pelo script fornecido, herança padrão mantida e conta Windows comum com acesso à máquina. Chave gerada em arquivo e/ou logs do seed presentes. ACL de uma instalação real não foi inspecionada; é achado condicional do artefato de deploy.

Evidência

- Arquivo: escolaatenta-installer.iss:66-70
- Arquivo: src/EscolaAtenta.API/Program.cs:87-101
- Arquivo: src/EscolaAtenta.Infrastructure/Data/DatabaseSeeder.cs:100-105
- Arquivo: src/EscolaAtenta.API/appsettings.json:12-12
- Revisão: f7550204891aa6f2914a9280fe26ddb2d55ef793

```text

Name: "{app}"; Permissions: admins-full system-full users-readexec

json["Jwt"]!["SecretKey"] = secretKey;

```

Verificação: Evidência estática do script e dos destinos de gravação; sem instalação, UAC ou alteração de ACL na máquina. Sem comprovação dinâmica de uma instalação específica.

Impacto

Leitura da chave de assinatura por usuário local permite forjar identidade administrativa; logs podem expor credencial inicial. A aleatoriedade da chave não protege contra sua leitura.

Sugestão de correção

Separar dados, segredos e logs dos binários. Restringir leitura ao serviço/SYSTEM e administradores. Dar ao TrayMonitor somente a informação operacional necessária, sem acesso à chave JWT. Provisionar a senha inicial por canal restrito e temporário.

Critérios de aceite

- [] Reproduzir o cenário descrito e confirmar a rejeição ou segregação esperada.
- [] Preservar o fluxo legítimo para o papel e os vínculos autorizados.
- [] Adicionar teste de regressão que use os controles reais e SQLite quando aplicável.

--- FIM ISSUE 8 ---